

EVIDENCE-BACKED STATIC REPOSITORY ASSESSMENT

andrewyng/openworker code intelligence report

Prioritized source, dependency, workflow, architecture, maintainability, and repository-governance evidence with explicit confidence, applicability, remediation, verification, and claim boundaries.



EXACT TARGET 602415a52adf · HEAD	EVIDENCE DISCIPLINE 100% traceable finding coverage	REPORT CONFIDENCE 100/100 · high
--	---	--

- 397 analyzed files
- 75,475 code LOC
- 37 findings
- clean working tree · fetched history

OVERALL SCORE 77/100 Good with targeted work	SECURITY 85/100 source + dependency + workflow signals	EFFORT RANGE 35–91 days engineering-only planning range	FINANCIAL IMPACT Not modeled A defensible dollar estimate requires organization-specific labor rates, incident history, delivery volume, and change-failure data.
--	--	---	---

Executive Summary

What the Evidence Says

The repository scores 77/100 (Good with targeted work) with high report confidence (100/100).

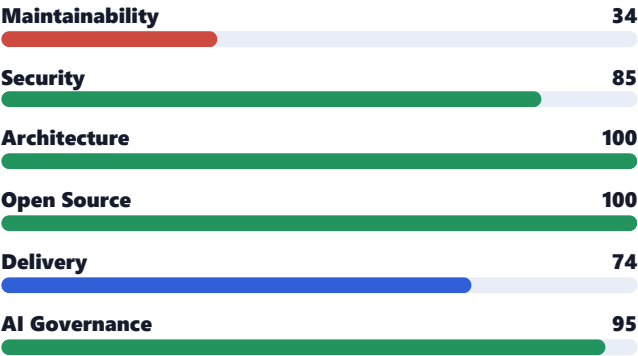
100% of findings contain traceable rule and source or repository evidence; 0 have a confirmed owner and 37 remain unassigned.

No dependency vulnerability rows were surfaced; package-manager audit status and lockfile evidence determine whether this is a verified clean result or a coverage gap.

Repository evidence mapping contains 1 material gap category/categories; this is an alignment signal, not a certification or control-effectiveness conclusion.

The applicable critical/high/medium queue implies roughly 35–91 engineering days before discovery, release coordination, and production validation.

Score Dimensions



EVIDENCE COVERAGE

100%

37/37 findings

CONFIRMED OWNERS

0

37 unassigned

DEPENDENCY VULNS

0

0 critical · 0 high

WORKFLOW GAPS

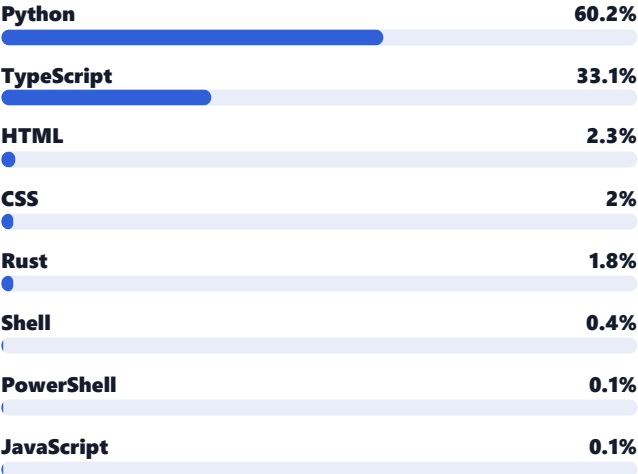
0

mutable action refs + implicit permission policies

Severity / Effort Distribution

	S	M	L	XL
CRITICAL	0	0	0	0
HIGH	0	0	10	0
MEDIUM	1	11	7	0
LOW	4	4	0	0

Codebase Composition



The overall score is a prioritization model. It is not a certification, exploitability determination, valuation opinion, productivity measurement, or substitute for runtime testing and human review.

PROVENANCE-LOCKED COMPARISON

Before / After Evidence Delta

BASELINE SCORE

69/100

db93d75bf634

CURRENT SCORE

77/100

602415a52adf

SCORE CHANGE

+8

same scanner source + settings

RESOLVED EVIDENCE

8

0 introduced · 37 still observed

Score Dimension Movement

DIMENSION	BEFORE	AFTER	CHANGE
Maintainability	33	34	+1
Security	63	85	+22
Architecture	100	100	0
Open Source	91	100	+9
Delivery	73	74	+1
AI Governance	69	95	+26

Observed Evidence Movement

SIGNAL	BEFORE	AFTER	CHANGE
All findings	45	37	-8
Critical + high findings	11	10	-1
Dependency vulnerabilities	1	0	-1
Mutable workflow action refs	15	0	-15
Implicit workflow permissions	2	0	-2
Repository governance gaps	6	1	-5

Priority Rule Evidence No Longer Observed

SEVERITY	RULE EVIDENCE	BASELINE LOCATION
high	Dependency vulnerability: xlsx clearcode.security.dependency-vulnerability-xlsx	surfaces/gui/package.json:1
medium	Dependency automation missing clearcode.open-source.dependency-automation-missing	.github/dependabot.yml:1
medium	GitHub Actions use mutable references clearcode.security.github-actions-use-mutable-references	.github/workflows/ci.yml:12
medium	Security policy missing clearcode.security.security-policy-missing	SECURITY.md:1
low	EditorConfig missing clearcode.maintainability.editorconfig-missing	.editorconfig:1
low	Git attributes missing clearcode.delivery.git-attributes-missing	.gitattributes:1
low	Workflow token permissions are implicit clearcode.security.workflow-token-permissions-are-implicit	.github/workflows/ci.yml:1
low	Contribution guide missing clearcode.delivery.contribution-guide-missing	CONTRIBUTING.md:1

Baseline: db93d75bf634e3a855b29e00d8f5d677438cac1f · Current: 602415a52adfc1adb762b85cc55bde3a2df24242

Scanner digest: fad562fefa9f8da41755117e7a4c66a5d530d45e3fa8e7e41f1fc8fd6760baac

This comparison describes scanner-observed repository evidence at two exact commits produced by the same scanner source and settings. A resolved finding means the rule evidence is absent in the current scan; it is not proof of runtime security, incident prevention, productivity, or financial value.

Methodology, Scope, and Claim Boundaries

Exact Audit Target

Repository: <https://github.com/andrewyng/openworker>

Commit: [602415a52adfc1adb762b85cc55bde3a2df24242](#) (July 25, 2026)

Branch: HEAD

Snapshot state: working tree clean; fetched history.

File discovery: Git tracked and non-ignored untracked files.

Generated: July 25, 2026; scanner runtime 25,762 ms.

Coverage and Traceability

SCAN COVERAGE

89.6%

397/443 candidate files

CODE LOC

75,475

82,589 analyzed text lines

EVIDENCE COVERAGE

100%

37 findings with traceable evidence

REPORT CONFIDENCE

100 / 100

high confidence

Repository Scope Classification

SCOPE	FILES	HOW FINDINGS ARE INTERPRETED
Production	225	Directly applicable to shipped or runtime code.
Build / delivery control	6	Applies to CI, release, packaging, or infrastructure controls.
Tests and fixtures	156	Contextual evidence; severity is reduced unless production reachability is established.
Examples and documentation	10	Advisory or informational; excluded from direct production-risk claims.
Generated report artifacts	0	Excluded from the report's own finding evidence to prevent self-contamination.

Analysis Procedure

- Enumerate text candidates with size, directory, ignore-file, and generated-artifact limits.
- Classify production, control-plane, test, fixture, example, documentation, and generated scope.
- Extract language-aware units and rule evidence; attach rule IDs, fingerprints, confidence, applicability, remediation, and verification.
- Inspect dependency manifests, lock evidence, package-manager audit output, workflow permissions, and immutable action provenance.
- Score with confidence/applicability weighting and diminishing penalties; do not infer dollar impact or historical trends from a single scan.

Limitations

Static repository analysis cannot prove runtime exploitability, production reachability, or control operation.

Dependency audit coverage depends on available package-manager tooling, network access, and resolvable manifests.

Git history metrics reflect the fetched history window, not necessarily the repository's full lifetime.

Framework mappings are evidence-alignment aids, not certification, attestation, or compliance determinations.

Dependency audit status: 2/2 applicable audit command(s) completed.

Project Profile and Stack Coverage

Detected Project Shape

The scan detected 397 analyzed files, 75,475 lines of code, 4 dependency or build manifest(s), and 8 language signal(s).

JavaScript/TypeScript application or package workspace

Python service, automation, or data/AI codebase

Rust systems, CLI, or backend component

multi-component repository requiring ownership mapping

This page tunes the audit interpretation by stack. A Node/TypeScript repo needs different evidence than a Go service, Ruby app, Rust CLI, JVM service, or .NET system. The remediation plan should preserve proof that fits the language and package ecosystem actually present.

Coverage Signals

ECOSYSTEMS

3

Python, Node.js, Rust

MANIFESTS

4

dependency/build files

LOCK GAPS

0

manifest(s) without lock evidence

LANGUAGES

8

detected code families

Language-Specific Review Focus

LANGUAGE	DEPENDENCY EVIDENCE	AUDIT FOCUS
Python 60.2% 45,454 LOC	Python, Node.js, Rust	requirements/pyproject reproducibility, runtime pinning, data/AI workflow validation, and test evidence.
TypeScript 33.1% 24,976 LOC	Python, Node.js, Rust	package lock integrity, type-safety boundaries, SDK/API drift, test evidence, and dependency reachability.
HTML 2.3% 1,749 LOC	Python, Node.js, Rust	Maintainability, ownership, dependency evidence, and test confidence for this stack.
CSS 2% 1,526 LOC	Python, Node.js, Rust	Maintainability, ownership, dependency evidence, and test confidence for this stack.
Rust 1.8% 1,341 LOC	Python, Node.js, Rust	Cargo.lock policy, crate ownership, unsafe/native boundaries, and release confidence.
Shell 0.4% 276 LOC	Python, Node.js, Rust	Maintainability, ownership, dependency evidence, and test confidence for this stack.

Manifest and Reproducibility Evidence

MANIFEST	ECOSYSTEM	DEPENDENCIES	REPRODUCIBILITY EVIDENCE
pyproject.toml	Python	6	lock/equivalent present
surfaces/gui/package.json	Node.js	21	lock/equivalent present
surfaces/gui/src-tauri/Cargo.toml	Rust	10	lock/equivalent present
stt/Cargo.toml	Rust	5	lock/equivalent present

Priority Action Plan

PRIORITY QUEUE

6

highest-confidence applicable findings

EFFORT RANGE

35–91 days

engineering-only estimate

NAMED OWNERS

0

37 findings remain unassigned

VERIFICATION SPECS

100%

remediation plus verification present

Recommended Sequencing

Contain: patch exploitable dependencies and tighten delivery controls before broad refactoring.

Reduce change risk: split the highest-complexity production units behind tests and stable interfaces.

Institutionalize: assign real owners, record acceptance criteria, and retain verification evidence from the next scan.

The effort range is a planning aid derived from finding size and applicability. It excludes discovery, release windows, stakeholder coordination, and production validation.

Highest-Priority Backlog

RISK	EVIDENCE	REQUIRED CHANGE	PROOF OF COMPLETION
HIGH P1 · L effort high confidence	Function complexity hotspot: App surfaces/gui/src/App.tsx:146 clearcode.maintainability.function-complexity-hotspot-app · direct App spans 1455 non-empty lines with 401 decision token(s) and a 74.3/100 modeled complexity score.	Split App around explicit policy, state, or I/O boundaries while preserving its public contract; add focused tests for each extracted path.	Run the affected test suite and re-scan surfaces/gui/src/App.tsx:146 ; verify the function's decision count and modeled complexity both decrease without moving complexity into an untested helper.
HIGH P2 · L effort high confidence	Function complexity hotspot: Composer surfaces/gui/src/components/Composer.tsx:82 clearcode.maintainability.function-complexity-hotspot-composer · direct Composer spans 434 non-empty lines with 144 decision token(s) and a 60.8/100 modeled complexity score.	Split Composer around explicit policy, state, or I/O boundaries while preserving its public contract; add focused tests for each extracted path.	Run the affected test suite and re-scan surfaces/gui/src/components/Composer.tsx:82 ; verify the function's decision count and modeled complexity both decrease without moving complexity into an untested helper.
HIGH P3 · L effort high confidence	Function complexity hotspot: Sidebar surfaces/gui/src/components/Sidebar.tsx:173 clearcode.maintainability.function-complexity-hotspot-sidebar · direct Sidebar spans 1083 non-empty lines with 204 decision token(s) and a 69.1/100 modeled complexity score.	Split Sidebar around explicit policy, state, or I/O boundaries while preserving its public contract; add focused tests for each extracted path.	Run the affected test suite and re-scan surfaces/gui/src/components/Sidebar.tsx:173 ; verify the function's decision count and modeled complexity both decrease without moving complexity into an untested helper.
HIGH P4 · L effort high confidence	Large file hotspot surfaces/gui/src/api.ts:1 clearcode.maintainability.large-file-hotspot · direct 1673 non-empty lines make ownership, testing, and refactoring harder.	Refactor the hotspot into a smaller, named unit and verify behavior with focused tests or checks.	Re-run the scan and review surfaces/gui/src/api.ts:1 for the corrected pattern.
HIGH P5 · L effort high confidence	Large file hotspot surfaces/gui/src/App.tsx:1 clearcode.maintainability.large-file-hotspot · direct 1674 non-empty lines make ownership, testing, and refactoring harder.	Refactor the hotspot into a smaller, named unit and verify behavior with focused tests or checks.	Re-run the scan and review surfaces/gui/src/App.tsx:1 for the corrected pattern.
HIGH P6 · L effort high confidence	Large file hotspot surfaces/gui/src/components/Sidebar.tsx:1 clearcode.maintainability.large-file-hotspot · direct 1335 non-empty lines make ownership, testing, and refactoring harder.	Refactor the hotspot into a smaller, named unit and verify behavior with focused tests or checks.	Re-run the scan and review surfaces/gui/src/components/Sidebar.tsx:1 for the corrected pattern.

Security, Dependencies, and Delivery Controls

Current Risk Baseline

No historical trend series was available in this scan payload, so this page treats the current scan as the baseline for the next remediation cycle.

The baseline is still actionable: 37 unassigned finding(s), 1 repository-governance gap(s), and 1 framework evidence gap category/categories.

The applicable queue implies 35–91 engineering days. The next comparable scan should show finding closure, retained evidence, assigned ownership, and verified dependency/workflow controls.

Risk Signals to Track

SIGNAL	CURRENT VALUE	WHY IT MATTERS
Maintainability debt	34 / 100 high	Prioritize high-risk functions and large files, then convert repetitive patterns into named helpers.
AI-assisted change context pressure	High relative pressure (100/100) high	Instrument comparable tasks, reduce context sprawl, add verification proof, and compare measured before/after telemetry.
Framework evidence gaps	73 / 100 high	Attach evidence to each mapped theme, close material gaps, and keep repository policies versioned.
Ownership concentration debt	93% top contributor share medium	Document ownership, rotate reviews, and move implicit operational knowledge into runbooks.
Security debt	85 / 100 low	Resolve critical/high issues first, preserve audit output, and document residual accepted risk.

DEPENDENCIES

42

4 manifests

RISKY SPECS

0

unpinned or loose versions

SECURITY FINDINGS

0

0 critical, 0 high

FIXABLE VULNS

0

package-manager remediable

Dependency Audit Evidence

SOURCE	STATUS / SEVERITY	PACKAGE	EVIDENCE
npm-audit	ok	surfaces/gui/package.json	Dependency governance signal from scan.
pip-audit	ok	pyproject.toml	Dependency governance signal from scan.

GitHub Actions Security Evidence

Workflow permissions, trigger privilege, expression interpolation, and action-reference provenance are evaluated independently from application source. Mutable tags and implicit permission defaults are supply-chain control gaps even when no exploit path is confirmed by static analysis.

GitHub Actions Security Evidence

WORKFLOWS
2
analyzed YAML files

EXPLICIT PERMISSIONS
2
0 implicit

IMMUTABLE ACTIONS
15
0 mutable refs

PRIVILEGED PR
0
pull_request_target workflows

WORKFLOW	PERMISSIONS	ACTION PROVENANCE	PRIVILEGED / INJECTION SIGNALS
.github/workflows/ci.yml	explicit	6 immutable / 0 mutable	none detected
.github/workflows/release.yml	explicit	9 immutable / 0 mutable	none detected

Architecture and Maintainability

Architecture Map



Maintainability Narrative

The repository scores 77/100 (Good with targeted work) with high report confidence (100/100).

100% of findings contain traceable rule and source or repository evidence; 0 have a confirmed owner and 37 remain unassigned.

No dependency vulnerability rows were surfaced; package-manager audit status and lockfile evidence determine whether this is a verified clean result or a coverage gap.

Repository evidence mapping contains 1 material gap category/categories; this is an alignment signal, not a certification or control-effectiveness conclusion.

The applicable critical/high/medium queue implies roughly 35–91 engineering days before discovery, release coordination, and production validation.

Coupling and Complexity Signals

	S	M	L	XL
CRITICAL	0	0	0	0
HIGH	0	0	10	0
MEDIUM	1	11	7	0
LOW	4	4	0	0

AI-Assisted Change Measurement

Relative Context-Pressure Index

100 / 100 (High)

High relative context pressure: repository signals indicate where AI-assisted changes may require broader search, more reasoning, and stronger validation.

This is a comparative repository heuristic. It is not a measured token count, productivity claim, or provider-cost estimate. ClearCode intentionally withholds dollar burn until real task telemetry and organization-specific pricing are supplied.

Evidence Drivers

DRIVER	POINTS	EVIDENCE INTERPRETATION
Complexity drag 60.8 p95 complexity	73.3	Dense control flow increases reasoning depth, diff size, and rewrite attempts.
Large-context files 14 large file(s)	71.5	Large and dense files consume prompt budget before the agent reaches the change point.
Deferred decisions 13 file(s) with deferred-work markers	23.4	Inline deferred-work notes and suppressions force agents to decide whether a path is intentional, stale, or unsafe.
Tribal knowledge 93% top contributor share	9.5	Concentrated ownership means more implicit context is missing from the repository itself.

Telemetry Required for Defensible Before/After Claims

MEASURE	COLLECTION POINT	DECISION IT SUPPORTS
Input/output tokens by task	Agent or API usage logs, grouped by repository task type	Whether the same class of change consumes less context after remediation.
Search and retry count	Tool-call traces and failed patch attempts	Whether boundaries, tests, and documentation reduce reasoning loops.
Human review minutes and cycles	Pull-request review events	Whether proposed changes become easier to validate safely.
Change-failure and rollback rate	CI, deployment, incident, and revert telemetry	Whether speed improvements preserve reliability.
Cost	Measured tokens × contracted provider rates, plus review labor inputs	Only then calculate an organization-specific operating-cost delta.

AI Change Hotspots and Instrumentation Plan

These files rank highest on the relative context-pressure heuristic. Instrument comparable change tasks here first, then compare token usage, retries, review cycles, and change-failure outcomes before and after remediation. The ranking does not estimate spend or individual productivity.

Repository Hotspots to Instrument First

FILE	RELATIVE PRESSURE	OBSERVED CAUSES
surfaces/gui/src-tauri/src/lib.rs	100 / 100	711 LOC, 89 branch token(s), 9 finding weight
surfaces/gui/src/api.ts	100 / 100	1673 LOC, 258 branch token(s), 4 finding weight, 1869 recent churn
surfaces/gui/src/App.tsx	100 / 100	1674 LOC, 349 branch token(s), 3 deferred marker(s), 8 finding weight, 1788 recent churn
surfaces/gui/src/humanize.ts	100 / 100	82 branch token(s), 1 deferred marker(s), 3 finding weight
surfaces/gui/src/styles.css	100 / 100	1521 LOC, 31 dense line(s), 8 deferred marker(s), 9 finding weight, 1610 recent churn
surfaces/gui/src/providers/ProviderSetup.tsx	100 / 100	87 branch token(s)
surfaces/gui/src/components/AccessSection.tsx	100 / 100	76 branch token(s)
surfaces/gui/src/components/ApprovalCard.tsx	100 / 100	68 branch token(s)

Minimum Viable Measurement Design

- Baseline:** select recurring task classes and record at least three comparable observations per class.
- Intervention:** apply one bounded change at a time—such as module extraction, targeted tests, or dependency isolation.
- Comparison:** evaluate medians and ranges, preserve failure cases, and reject conclusions when task scope differs materially.
- Guardrail:** pair speed and context measures with review quality, rollback, and change-failure evidence.

Technical Debt Register

This register groups evidence by engineering risk domain. Each row connects an observed signal to its likely change-risk implication and a concrete next action; it does not assign financial impact.

DEBT CATEGORY	SIGNAL	SEVERITY	EVIDENCE	COMMERCIAL IMPLICATION	RECOMMENDED ACTION
Maintainability debt	34 / 100	high	31 maintainability finding(s). 60.8 p95 complexity. 32.3% large-unit share.	Higher maintenance cost, slower onboarding, and greater regression risk during feature work.	Prioritize high-risk functions and large files, then convert repetitive patterns into named helpers.
AI-assisted change context pressure	High relative pressure (100/100)	high	4 active repository driver(s). No token, time, productivity, or dollar value is inferred without measured task telemetry.	Complexity, weak verification, and unclear boundaries can increase search, retry, and validation work for AI-assisted changes.	Instrument comparable tasks, reduce context sprawl, add verification proof, and compare measured before/after telemetry.
Framework evidence gaps	73 / 100	high	2 strong framework signal(s). 3 partial signal(s). 1 evidence gap(s).	Missing repository evidence increases uncertainty during security, quality, and change-control review.	Attach evidence to each mapped theme, close material gaps, and keep repository policies versioned.
Ownership concentration debt	93% top contributor share	medium	5 contributor(s). 60 recent commit(s). 15 git hotspot(s). Ownership inference threshold met.	Knowledge concentration creates key-person risk during handoff and enterprise support.	Document ownership, rotate reviews, and move implicit operational knowledge into runbooks.
Security debt	85 / 100	low	2 security finding(s). 0 dependency vulnerability signal(s). 0 fixable by package-manager workflow.	Security findings increase exploitability uncertainty, patch urgency, and release-review effort.	Resolve critical/high issues first, preserve audit output, and document residual accepted risk.
Architecture debt	100 / 100	low	15 components. 2 dependency edges. 0 cycle(s).	Coupling increases integration risk, test surface, and cost of changing core workflows.	Break cyclic dependencies, isolate cross-cutting modules, and define ownership boundaries.
Dependency governance debt	42 dependencies	low	0 risky version spec(s). 0 manifest(s) without lockfile. 0 missing license signal(s).	Unclear dependency provenance weakens technical confidence and increases supply-chain exposure.	Pin production dependencies, keep lockfiles, record licenses, and enable dependency update automation.
Delivery and testing debt	42.3% test ratio	low	4 delivery finding(s). 0 reliability finding(s). 1 safe fix(es) currently available.	Weak verification coverage turns routine fixes into delivery-risking delays.	Add smoke tests for core workflows, then expand regression coverage around risky code paths.
AI-assisted change governance debt	100% evidence coverage	low	37 rule-backed finding(s). 0 suppressed finding(s). 42.3% test ratio. 0 dependency vulnerability signal(s).	AI-assisted delivery increases review pressure when changes are not tied to ownership, verification, and post-merge evidence.	Track review cycle time, rewrite rate, rollback rate, owner clarity, boundary drift, and failure-mode test coverage after AI-assisted changes.

Framework Evidence Mapping

ALIGNMENT INDEX
73/100
weighted repository evidence proxy

STRONG SIGNALS
2
80–100 evidence score

PARTIAL SIGNALS
3
60–79 evidence score

EVIDENCE GAPS
1
below 60 evidence score

These scores map visible repository artifacts to selected framework themes. They do not establish organizational process maturity, operating effectiveness, legal compliance, certification readiness, or audit assurance.

FRAMEWORK THEME	CATEGORY	SIGNAL	REPOSITORY EVIDENCE	PRIORITY ACTION
ISO 9001 ISO 9001:2015 Repository Evidence Signals	Quality process	76/100	Automated tests are present.	Create CI/test gates
ISO 27001 ISO/IEC 27001 Repository Evidence Signals	Security management	86/100	Security disclosure policy is present.	Add security policy
ISO 25010 ISO/IEC 25010 Product-Quality Evidence Signals	Product quality	84/100	Architecture score is 100/100.	Reduce red-zone functions
ISO 42001 ISO/IEC 42001 AI-Governance Evidence Signals	AI governance	62/100	AI governance score is 95/100.	Require CI for AI changes
SOC 2 SOC 2 Repository Evidence Signals	Trust controls	72/100	Security policy evidence is present.	Collect security/change-control evidence
Safety High-Assurance Repository Evidence Signals	High assurance	58/100	Automated tests are present.	Add traceable verification gates

Prioritized Findings Ledger

Findings are ordered by severity, then confidence and applicability. Test, fixture, example, and documentation signals are labeled so they cannot silently inherit production-risk claims.

RISK	FINDING AND LOCATION	EVIDENCE QUALITY	REMEDIATION / VERIFICATION
HIGH L effort	Large file hotspot surfaces/gui/src/api.ts:1 clearcode.maintainability.large-file-hotspot 1673 non-empty lines make ownership, testing, and refactoring harder.	high confidence direct applicability production scope	Change: Refactor the hotspot into a smaller, named unit and verify behavior with focused tests or checks. Proof: Re-run the scan and review surfaces/gui/src/api.ts:1 for the corrected pattern.
HIGH L effort	Large file hotspot surfaces/gui/src/App.tsx:1 clearcode.maintainability.large-file-hotspot 1674 non-empty lines make ownership, testing, and refactoring harder.	high confidence direct applicability production scope	Change: Refactor the hotspot into a smaller, named unit and verify behavior with focused tests or checks. Proof: Re-run the scan and review surfaces/gui/src/App.tsx:1 for the corrected pattern.
HIGH L effort	Large file hotspot surfaces/gui/src/components/Sidebar.tsx:1 clearcode.maintainability.large-file-hotspot 1335 non-empty lines make ownership, testing, and refactoring harder.	high confidence direct applicability production scope	Change: Refactor the hotspot into a smaller, named unit and verify behavior with focused tests or checks. Proof: Re-run the scan and review surfaces/gui/src/components/Sidebar.tsx:1 for the corrected pattern.
HIGH L effort	Large file hotspot coworker/server/app.py:1 clearcode.maintainability.large-file-hotspot 1748 non-empty lines make ownership, testing, and refactoring harder.	high confidence direct applicability production scope	Change: Refactor the hotspot into a smaller, named unit and verify behavior with focused tests or checks. Proof: Re-run the scan and review coworker/server/app.py:1 for the corrected pattern.
HIGH L effort	Large file hotspot coworker/server/manager.py:1 clearcode.maintainability.large-file-hotspot 3490 non-empty lines make ownership, testing, and refactoring harder.	high confidence direct applicability production scope	Change: Refactor the hotspot into a smaller, named unit and verify behavior with focused tests or checks. Proof: Re-run the scan and review coworker/server/manager.py:1 for the corrected pattern.
HIGH L effort	Large file hotspot coworker/connectors/descriptors.py:1 clearcode.maintainability.large-file-hotspot 1385 non-empty lines make ownership, testing, and refactoring harder.	high confidence direct applicability production scope	Change: Refactor the hotspot into a smaller, named unit and verify behavior with focused tests or checks. Proof: Re-run the scan and review coworker/connectors/descriptors.py:1 for the corrected pattern.
HIGH L effort	Large file hotspot coworker/connectors/integration_tools.py:1 clearcode.maintainability.large-file-hotspot 4516 non-empty lines make ownership, testing, and refactoring harder.	high confidence direct applicability production scope	Change: Refactor the hotspot into a smaller, named unit and verify behavior with focused tests or checks. Proof: Re-run the scan and review coworker/connectors/integration_tools.py:1 for the corrected pattern.
HIGH L effort	Function complexity hotspot: App surfaces/gui/src/App.tsx:146 clearcode.maintainability.function-complexity-hotspot-app App spans 1455 non-empty lines with 401 decision token(s) and a 74.3/100 modeled complexity score.	high confidence direct applicability production scope	Change: Split App around explicit policy, state, or I/O boundaries while preserving its public contract; add focused tests for each extracted path. Proof: Run the affected test suite and re-scan surfaces/gui/src/App.tsx:146 ; verify the function's decision count and modeled complexity both decrease without moving complexity into an untested helper.
HIGH L effort	Function complexity hotspot: Sidebar surfaces/gui/src/components/Sidebar.tsx:173 clearcode.maintainability.function-complexity-hotspot-sidebar Sidebar spans 1083 non-empty lines with 204 decision token(s) and a 69.1/100 modeled complexity score.	high confidence direct applicability production scope	Change: Split Sidebar around explicit policy, state, or I/O boundaries while preserving its public contract; add focused tests for each extracted path. Proof: Run the affected test suite and re-scan surfaces/gui/src/components/Sidebar.tsx:173 ; verify the function's decision count and modeled complexity both decrease without moving complexity into an untested helper.
HIGH L effort	Function complexity hotspot: Composer surfaces/gui/src/components/Composer.tsx:82 clearcode.maintainability.function-complexity-hotspot-composer Composer spans 434 non-empty lines with 144 decision token(s) and a 60.8/100 modeled complexity score.	high confidence direct applicability production scope	Change: Split Composer around explicit policy, state, or I/O boundaries while preserving its public contract; add focused tests for each extracted path. Proof: Run the affected test suite and re-scan surfaces/gui/src/components/Composer.tsx:82 ; verify the function's decision count and modeled complexity both decrease without moving complexity into an untested helper.

RISK	FINDING AND LOCATION	EVIDENCE QUALITY	REMEDIATION / VERIFICATION
MEDIUM M effort	Deferred work concentration surfaces/gui/src/styles.css:558 clearcode.delivery.deferred-work-concentration 8 TODO/FIXME/HACK markers indicate unresolved decisions in one file.	medium confidence direct applicability production scope	Change: Refactor the hotspot into a smaller, named unit and verify behavior with focused tests or checks. Proof: Re-run the scan and review surfaces/gui/src/styles.css:558 for the corrected pattern.
MEDIUM M effort	Deferred work concentration surfaces/gui/src/components/RightRail.tsx:44 clearcode.delivery.deferred-work-concentration 9 TODO/FIXME/HACK markers indicate unresolved decisions in one file.	medium confidence direct applicability production scope	Change: Refactor the hotspot into a smaller, named unit and verify behavior with focused tests or checks. Proof: Re-run the scan and review surfaces/gui/src/components/RightRail.tsx:44 for the corrected pattern.
MEDIUM M effort	Contributor concentration risk Repository clearcode.delivery.contributor-concentration-risk 93% of recent commits came from one contributor.	medium confidence review applicability repository scope	Change: Refactor the hotspot into a smaller, named unit and verify behavior with focused tests or checks. Proof: Re-run the scan and confirm the repository-level signal improves.
MEDIUM L effort	Large file hotspot surfaces/gui/src-tauri/src/lib.rs:1 clearcode.maintainability.large-file-hotspot 711 non-empty lines make ownership, testing, and refactoring harder.	high confidence direct applicability production scope	Change: Refactor the hotspot into a smaller, named unit and verify behavior with focused tests or checks. Proof: Re-run the scan and review surfaces/gui/src-tauri/src/lib.rs:1 for the corrected pattern.
MEDIUM M effort	Dense lines reduce reviewability surfaces/gui/src/styles.css:185 clearcode.maintainability.dense-lines-reduce-reviewability 31 lines exceed 140 characters, increasing scan and review effort.	high confidence direct applicability production scope	Change: Refactor the hotspot into a smaller, named unit and verify behavior with focused tests or checks. Proof: Re-run the scan and review surfaces/gui/src/styles.css:185 for the corrected pattern.
MEDIUM L effort	Large declarative file surfaces/gui/src/styles.css:1 clearcode.maintainability.large-declarative-file 1521 non-empty lines concentrate a large declarative surface; review modularity and ownership before splitting it mechanically.	medium confidence direct applicability production scope	Change: Refactor the hotspot into a smaller, named unit and verify behavior with focused tests or checks. Proof: Re-run the scan and review surfaces/gui/src/styles.css:1 for the corrected pattern.
MEDIUM L effort	Large file hotspot surfaces/gui/src/components/ManageTab.s.tsx:1 clearcode.maintainability.large-file-hotspot 851 non-empty lines make ownership, testing, and refactoring harder.	high confidence direct applicability production scope	Change: Refactor the hotspot into a smaller, named unit and verify behavior with focused tests or checks. Proof: Re-run the scan and review surfaces/gui/src/components/ManageTabs.tsx:1 for the corrected pattern.
MEDIUM M effort	Dense lines reduce reviewability surfaces/gui/src/components/SettingsView.w.tsx:270 clearcode.maintainability.dense-lines-reduce-reviewability 12 lines exceed 140 characters, increasing scan and review effort.	high confidence direct applicability production scope	Change: Refactor the hotspot into a smaller, named unit and verify behavior with focused tests or checks. Proof: Re-run the scan and review surfaces/gui/src/components/SettingsView.tsx:270 for the corrected pattern.
MEDIUM L effort	Large file hotspot surfaces/gui/src/components/SettingsView.w.tsx:1 clearcode.maintainability.large-file-hotspot 718 non-empty lines make ownership, testing, and refactoring harder.	high confidence direct applicability production scope	Change: Refactor the hotspot into a smaller, named unit and verify behavior with focused tests or checks. Proof: Re-run the scan and review surfaces/gui/src/components/SettingsView.tsx:1 for the corrected pattern.
MEDIUM L effort	Large file hotspot coworker/engine.py:1 clearcode.maintainability.large-file-hotspot 956 non-empty lines make ownership, testing, and refactoring harder.	high confidence direct applicability production scope	Change: Refactor the hotspot into a smaller, named unit and verify behavior with focused tests or checks. Proof: Re-run the scan and review coworker/engine.py:1 for the corrected pattern.
MEDIUM L effort	Large file hotspot coworker/connectors/email_tools.py:1 clearcode.maintainability.large-file-hotspot 765 non-empty lines make ownership, testing, and refactoring harder.	high confidence direct applicability production scope	Change: Refactor the hotspot into a smaller, named unit and verify behavior with focused tests or checks. Proof: Re-run the scan and review coworker/connectors/email_tools.py:1 for the corrected pattern.
MEDIUM L effort	Large file hotspot coworker/connectors/tool_defs.py:1 clearcode.maintainability.large-file-hotspot 1172 non-empty lines make ownership, testing, and refactoring harder.	high confidence direct applicability production scope	Change: Refactor the hotspot into a smaller, named unit and verify behavior with focused tests or checks. Proof: Re-run the scan and review coworker/connectors/tool_defs.py:1 for the corrected pattern.

RISK	FINDING AND LOCATION	EVIDENCE QUALITY	REMEDIATION / VERIFICATION
MEDIUM M effort	Function complexity hotspot: VoiceInputSection surfaces/gui/src/components/SettingsView.w.tsx:143 clearcode.maintainability.function-complexity-hotspot-voiceinputsection VoiceInputSection spans 193 non-empty lines with 77 decision token(s) and a 68/100 modeled complexity score.	high confidence direct applicability production scope	Change: Split VoiceInputSection around explicit policy, state, or I/O boundaries while preserving its public contract; add focused tests for each extracted path. Proof: Run the affected test suite and re-scan surfaces/gui/src/components/SettingsView.tsx:143 ; verify the function's decision count and modeled complexity both decrease without moving complexity into an untested helper.
MEDIUM M effort	Function complexity hotspot: run surfaces/gui/src-tauri/src/lib.rs:580 clearcode.maintainability.function-complexity-hotspot-run run spans 186 non-empty lines with 23 decision token(s) and a 64.5/100 modeled complexity score.	high confidence direct applicability production scope	Change: Split run around explicit policy, state, or I/O boundaries while preserving its public contract; add focused tests for each extracted path. Proof: Run the affected test suite and re-scan surfaces/gui/src-tauri/src/lib.rs:580 ; verify the function's decision count and modeled complexity both decrease without moving complexity into an untested helper.
MEDIUM M effort	Function complexity hotspot: humanizeTool surfaces/gui/src/humanize.ts:29 clearcode.maintainability.function-complexity-hotspot-humanizetool humanizeTool spans 74 non-empty lines with 60 decision token(s) and a 56.9/100 modeled complexity score.	high confidence direct applicability production scope	Change: Split humanizeTool around explicit policy, state, or I/O boundaries while preserving its public contract; add focused tests for each extracted path. Proof: Run the affected test suite and re-scan surfaces/gui/src/humanize.ts:29 ; verify the function's decision count and modeled complexity both decrease without moving complexity into an untested helper.
MEDIUM M effort	Function complexity hotspot: InboxRoutingCard surfaces/gui/src/components/InboxConfigure.tsx:56 clearcode.maintainability.function-complexity-hotspot-inboxroutingcard InboxRoutingCard spans 100 non-empty lines with 34 decision token(s) and a 56/100 modeled complexity score.	high confidence direct applicability production scope	Change: Split InboxRoutingCard around explicit policy, state, or I/O boundaries while preserving its public contract; add focused tests for each extracted path. Proof: Run the affected test suite and re-scan surfaces/gui/src/components/InboxConfigure.tsx:56 ; verify the function's decision count and modeled complexity both decrease without moving complexity into an untested helper.
MEDIUM M effort	Function complexity hotspot: ModelsTab surfaces/gui/src/components/ManageTabs.tsx:78 clearcode.maintainability.function-complexity-hotspot-modelstab ModelsTab spans 89 non-empty lines with 24 decision token(s) and a 56/100 modeled complexity score.	high confidence direct applicability production scope	Change: Split ModelsTab around explicit policy, state, or I/O boundaries while preserving its public contract; add focused tests for each extracted path. Proof: Run the affected test suite and re-scan surfaces/gui/src/components/ManageTabs.tsx:78 ; verify the function's decision count and modeled complexity both decrease without moving complexity into an untested helper.
MEDIUM M effort	Function complexity hotspot: itemsFromMessages surfaces/gui/src/itemsFromMessages.ts:11 clearcode.maintainability.function-complexity-hotspot-itemsfrommessages itemsFromMessages spans 71 non-empty lines with 37 decision token(s) and a 53.8/100 modeled complexity score.	high confidence direct applicability production scope	Change: Split itemsFromMessages around explicit policy, state, or I/O boundaries while preserving its public contract; add focused tests for each extracted path. Proof: Run the affected test suite and re-scan surfaces/gui/src/itemsFromMessages.ts:11 ; verify the function's decision count and modeled complexity both decrease without moving complexity into an untested helper.
MEDIUM S effort	Webview script evaluation boundary surfaces/gui/src-tauri/src/lib.rs:743 clearcode.security.webview-script-evaluation-boundary This execution or data-handling boundary requires exploitability review before release.	medium confidence direct applicability production scope	Change: Triage exploitability, remove the unsafe pattern, add a regression test, and preserve scan evidence. Proof: Re-run the scan and review surfaces/gui/src-tauri/src/lib.rs:743 for the corrected pattern.
LOW S effort	CODEOWNERS missing CODEOWNERS:1 clearcode.delivery.codeowners-missing Code ownership makes reviews and escalation paths explicit during integration work.	medium confidence control applicability repository scope	Change: Apply the available Create file automation and verify the diff. Proof: Re-run the ClearCode scan and confirm the automated change removed this finding without regressions.
LOW S effort	Duplicate implementation block tests/test_automation.py:299 clearcode.maintainability.duplicate-implementation-block 1 duplicate cluster(s), up to 5 repeated blocks across 1 component(s), increase change cost.	high confidence contextual applicability test scope	Change: Refactor the hotspot into a smaller, named unit and verify behavior with focused tests or checks. Proof: Re-run the scan and review tests/test_automation.py:299 for the corrected pattern.
LOW S effort	Duplicate implementation block tests/test_message_source.py:25 clearcode.maintainability.duplicate-implementation-block 1 duplicate cluster(s), up to 3 repeated blocks across 1 component(s), increase change cost.	high confidence contextual applicability test scope	Change: Refactor the hotspot into a smaller, named unit and verify behavior with focused tests or checks. Proof: Re-run the scan and review tests/test_message_source.py:25 for the corrected pattern.

RISK	FINDING AND LOCATION	EVIDENCE QUALITY	REMEDIATION / VERIFICATION
LOW M effort	Duplicate implementation block coworker/providers/anthropic_provider.py:24 clearcode.maintainability.duplicate-implementation-block 1 duplicate cluster(s), up to 3 repeated blocks across 1 component(s), increase change cost.	high confidence direct applicability production scope	Change: Refactor the hotspot into a smaller, named unit and verify behavior with focused tests or checks. Proof: Re-run the scan and review coworker/providers/anthropic_provider.py:24 for the corrected pattern.
LOW M effort	Duplicate implementation block coworker/connectors/browser_automation.py:29 clearcode.maintainability.duplicate-implementation-block 1 duplicate cluster(s), up to 3 repeated blocks across 1 component(s), increase change cost.	high confidence direct applicability production scope	Change: Refactor the hotspot into a smaller, named unit and verify behavior with focused tests or checks. Proof: Re-run the scan and review coworker/connectors/browser_automation.py:29 for the corrected pattern.
LOW M effort	Duplicate implementation block coworker/connectors/browser_automation.py:33 clearcode.maintainability.duplicate-implementation-block 1 duplicate cluster(s), up to 3 repeated blocks across 1 component(s), increase change cost.	high confidence direct applicability production scope	Change: Refactor the hotspot into a smaller, named unit and verify behavior with focused tests or checks. Proof: Re-run the scan and review coworker/connectors/browser_automation.py:33 for the corrected pattern.
LOW M effort	Duplicate implementation block coworker/connectors/browser_automation.py:34 clearcode.maintainability.duplicate-implementation-block 1 duplicate cluster(s), up to 3 repeated blocks across 1 component(s), increase change cost.	high confidence direct applicability production scope	Change: Refactor the hotspot into a smaller, named unit and verify behavior with focused tests or checks. Proof: Re-run the scan and review coworker/connectors/browser_automation.py:34 for the corrected pattern.
INFO S effort	Unsafe HTML injection boundary ui-mocks/redesign.html:840 clearcode.security.unsafe-html-injection-boundary 2 instances of this boundary were observed in the file. The pattern occurs in example or prototype. Confirm it cannot reach production inputs or packaged artifacts before treating it as exploitable.	medium confidence contextual applicability example scope	Change: Triage exploitability, remove the unsafe pattern, add a regression test, and preserve scan evidence. Proof: Re-run the scan and review ui-mocks/redesign.html:840 for the corrected pattern.

Code Evidence Snippets

Bounded excerpts preserve the exact source context behind representative priority findings. Rule ID, confidence, fingerprint, remediation, and verification remain attached to each excerpt.

Large file hotspot

L effort

surfaces/gui/src/api.ts:1 | Maintainability | high
clearcode.maintainability.large-file-hotspot | confidence high | fingerprint 007e3a52db12

```
1 import type { SessionInfo, WsEvent } from "../types";  
2
```

1673 non-empty lines make ownership, testing, and refactoring harder.

Remediation: Refactor the hotspot into a smaller, named unit and verify behavior with focused tests or checks.

Verification: Re-run the scan and review surfaces/gui/src/api.ts:1 for the corrected pattern.

Large file hotspot

L effort

surfaces/gui/src/App.tsx:1 | Maintainability | high
clearcode.maintainability.large-file-hotspot | confidence high | fingerprint 4578e657c1fb

```
1 import { useCallback, useEffect, useRef, useState, type PointerEvent } from "react";  
2 import {
```

1674 non-empty lines make ownership, testing, and refactoring harder.

Remediation: Refactor the hotspot into a smaller, named unit and verify behavior with focused tests or checks.

Verification: Re-run the scan and review surfaces/gui/src/App.tsx:1 for the corrected pattern.

Large file hotspot

L effort

surfaces/gui/src/components/Sidebar.tsx:1 | Maintainability | high
clearcode.maintainability.large-file-hotspot | confidence high | fingerprint d37151b531c9

```
1 import { useEffect, useMemo, useState, type ReactNode } from "react";  
2 import {
```

1335 non-empty lines make ownership, testing, and refactoring harder.

Remediation: Refactor the hotspot into a smaller, named unit and verify behavior with focused tests or checks.

Verification: Re-run the scan and review surfaces/gui/src/components/Sidebar.tsx:1 for the corrected pattern.

Large file hotspot

L effort

coworker/server/app.py:1 | Maintainability | high
clearcode.maintainability.large-file-hotspot | confidence high | fingerprint d77a8601f4f1

```
1 """FastAPI app – OpenAI-compatible endpoint + WS session API + REST.  
2
```

1748 non-empty lines make ownership, testing, and refactoring harder.

Remediation: Refactor the hotspot into a smaller, named unit and verify behavior with focused tests or checks.

Verification: Re-run the scan and review coworker/server/app.py:1 for the corrected pattern.

Framework Evidence Appendix

These scores map visible repository artifacts to selected framework themes. They do not establish organizational process maturity, operating effectiveness, legal compliance, certification readiness, or audit assurance.

ISO 9001:2015 Repository Evidence Signals

Evidence that the repository can be changed through repeatable, governed engineering process.

Evidence

Automated tests are present.
CI workflow is present.
Contribution workflow is documented.
Formatting and text normalization are governed.

Gaps / Actions

CODEOWNERS is missing.
10 critical/high finding(s) require management.
Create CI/test gates
Document contribution workflow

ISO/IEC 27001 Repository Evidence Signals

Repository-level security governance, vulnerability management, and exposure control signals.

Evidence

Security disclosure policy is present.
Dependency update automation is present.
No critical/high dependency vulnerabilities were detected.
Dependency audit returned no vulnerability entries.

Gaps / Actions

2 source-level security finding(s) require triage.
Add security policy
Enable dependency automation

ISO/IEC 25010 Product-Quality Evidence Signals

Software quality model coverage across maintainability, reliability, security, compatibility, and operability proxies.

Evidence

Architecture score is 100/100.
Security score is 85/100.
Tests support reliability validation.
CI supports repeatable verification.

Gaps / Actions

Maintainability score is 34/100.
Reduce red-zone functions
Strengthen test and CI evidence

Framework Evidence Appendix

These scores map visible repository artifacts to selected framework themes. They do not establish organizational process maturity, operating effectiveness, legal compliance, certification readiness, or audit assurance.

ISO/IEC 42001 AI-Governance Evidence Signals

Controls for AI-assisted repository changes: traceability, human oversight, security, and release discipline.

Evidence

- AI governance score is 95/100.
- 100% traceable finding evidence.
- Security disclosure path exists for AI-assisted changes.
- CI can verify AI-assisted changes.

Gaps / Actions

- Human review ownership is not explicit.
- 2 source-level security finding(s) remain.
- Require CI for AI changes
- Keep human ownership explicit

SOC 2 Repository Evidence Signals

Cloud trust readiness proxy for security, availability, change control, and evidence collection.

Evidence

- Security policy evidence is present.
- Change verification evidence is present.
- Dependency monitoring evidence is present.
- No dependency vulnerability entries were detected.

Gaps / Actions

- Approval ownership evidence is missing.
- Top contributor share is 93%, creating continuity risk.
- Collect security/change-control evidence
- Add owner approval gates

High-Assurance Repository Evidence Signals

Advisory proxy for stricter environments such as aviation, automotive, medical, and embedded software.

Evidence

- Automated tests are present.
- CI verification is present.
- No dependency cycles detected.
- Test ratio is 42.3%.

Gaps / Actions

- 10 critical/high finding(s) remain.
- Critical complex unit share is 9.4%, above high-assurance tolerance.
- Add traceable verification gates
- Lower function complexity

Framework Sources and Interpretation Guide

Claim Boundary

These scores map visible repository artifacts to selected framework themes. They do not establish organizational process maturity, operating effectiveness, legal compliance, certification readiness, or audit assurance.

A score is a prioritization aid, not a substitute for threat modeling, runtime testing, architectural review, or audit evidence from organizational processes.

Research Basis

PRIMARY SOURCE	HOW IT INFORMS THIS REPORT	URL
NIST Secure Software Development Framework (SP 800-218)	Secure development practice and verification framing	https://csrc.nist.gov/pubs/sp/800/218/final
OpenSSF Scorecard	Repository security heuristics and explicit false-positive caveats	https://github.com/ossf/scorecard
OWASP Software Assurance Maturity Model	Governance, design, implementation, verification, and operations evidence	https://owasp.org/www-project-samm/
NIST AI Risk Management Framework	Govern, map, measure, and manage framing for AI-enabled systems	https://www.nist.gov/itl/ai-risk-management-framework
OWASP Top 10 for Agentic Applications 2026	Agent execution, tool, identity, memory, and human-oversight threat framing	https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/

Confidence Interpretation

100% of findings include rule, fingerprint, impact, and source or repository evidence.

2/2 applicable package-manager audit(s) completed successfully.

Git history includes 60 fetched commit(s).

High: direct, rule-backed evidence with a specific source or repository control.

Medium: a credible heuristic requiring applicability review.

Low: weak or indirect evidence; do not prioritize without manual confirmation.

Applicability Interpretation

Direct: production-code behavior or maintainability risk.

Control: CI, build, release, workflow, or repository-policy risk.

Review: applicability is unresolved and requires manual inspection.

Contextual / advisory: test, fixture, example, or documentation evidence; excluded from direct production-risk claims.